在 CML 新增自訂 XRd vRouter KVM 節點

English · PDF：English · 繁體中文

本說明書示範如何下載 Cisco XRd vRouter 26.2.2、建置 KVM 磁碟，並在 Cisco Modeling Labs（CML）新增可啟動的自訂節點。完成後，可在 CML lab 中加入 `IOS XRd vRouter (e1000)`，登入 IOS XR CLI，並看到管理與 data plane 介面。

僅限實驗環境。 Guest 以 `--privileged` 且不使用 AppArmor 限制的方式啟動 XRd 容器，這不是正式環境的設計。請見第 10 節。

適用資料來自 CML 2.10.0 與 2.10.1 的自訂節點。新建磁碟仍需依第 6 節確認啟動結果。

1. 安裝結構與所需檔案

本 repository 的 KVM（Kernel-based Virtual Machine）版本，由 CML 直接啟動一台 Ubuntu VM，再由 VM 內的 Docker 執行 XRd vRouter。XRd vRouter 透過 VM 的 PCI 網卡使用 data plane 介面。

```
CML host
└── KVM guest: xrd-vr-e1000
    ├── Ubuntu 24.04 + Docker
    ├── Cisco XRd vRouter 26.2.2 container
    ├── e1000 NIC 1 -> MgmtEth0/RP0/CPU0/0
    ├── e1000 NIC 2 -> GigabitEthernet0/0/0/0
    ├── e1000 NIC 3 -> GigabitEthernet0/0/0/1
    └── e1000 NIC 4 -> GigabitEthernet0/0/0/2
```

CML 需要三個項目：

項目	本說明書使用的值	用途
Node Definition	`xrd-vr-e1000`	定義 CPU、RAM、網卡、主控台與開機設定檔
Image Definition	`xrd-vr-e1000-26-2-2`	將磁碟映像關聯到 Node Definition
Disk Image	`xrd-vr-e1000-26-2-2.qcow2`	包含 Ubuntu、Docker、XRd 映像與啟動服務

Golden disk 是事先建好的基底磁碟。CML 為每個節點建立自己的 overlay，節點的變更寫入 overlay。請保留登錄後的基底磁碟，避免影響使用它的節點。

本 repository 是自訂整合，Cisco 未提供或支援這個自訂 Node Definition。Cisco 軟體需依其授權條款取得與使用；CML 授權不會自動涵蓋另外下載的 XRd vRouter。本 repository 只提供腳本及定義檔，軟體封存檔與 golden disk 由安裝者自行準備。

1.1 Repository 內容

以下路徑均以 `xrd-vrouter-cml/` 為起點。

檔案或目錄	用途
build/build-golden.sh	在 Linux 建置主機產生獨立 qcow2 磁碟
build/user-data、build/meta-data	建置 VM 使用的 cloud-init 設定
build/setup.sh	安裝 XRd 映像及開機服務
build/stub.sh	開機時執行設定光碟上的 `guest-init.sh`
definitions/xrd-vr-e1000-node.yaml	匯入 CML 的 Node Definition
definitions/xrd-vr-e1000-26-2-2-image.yaml	建立 Image Definition 時使用的欄位值

檔案或目錄	用途
definitions/xrd-vr-vmxnet3-node.yaml、definitions/xrd-vr-vmxnet3-26-2-2-image.yaml	選用的 vmxnet3 網卡版本（5.5 節）
examples/	轉送測試用的三份設定（6.1 節）

`guest-init.sh` 已內嵌在 Node Definition。CML 開機時將它連同 `first-boot.cfg`、`xrd.env` 放入設定光碟，交給 VM 執行。`build/guest-init.sh` 是可供閱讀的同一份腳本。

1.2 執行位置與資源

位置	工作	需求
操作電腦	Cisco 下載、編輯 YAML、開啟 CML Web UI、傳送檔案	macOS、Windows 或 Linux 均可
Linux 建置主機	將 Cisco 軟體封存檔製作成 qcow2	x86-64、KVM、QEMU、可連線至 Ubuntu 套件庫
CML 主機	存放 qcow2 並執行自訂節點	可使用 `/dev/kvm`，有主機 SSH 與 `sudo` 權限
XR CLI	確認 XRd 版本與介面	節點啟動後從 CML Console 登入

建置 VM 使用 **4 vCPU、6144 MiB RAM**。完成後的每個 CML 節點預設使用 **2 vCPU、7168 MiB RAM**。兩者分別是建置階段與節點執行階段的資源。

建置磁碟的虛擬容量為 32 GiB，qcow2 實際占用量隨內容成長。建置時還會保留 Ubuntu cloud image、XRd 封存檔及獨立磁碟。請先用 `df -h /var/tmp` 確認空間，並保留成長餘裕。

2. 從 Cisco 下載 XRd vRouter

執行位置：操作電腦的瀏覽器。

1. 開啟 Cisco Software Download。此頁為訪客可存取的公開頁面，但下載需要 CCO 權限。
2. 確認下載路徑為 `Routers > Virtual Routers > IOS XRd > XRd vRouter`。
3. 選擇本指南對應的 Release `26.2.2`。
4. 下載 `xrd-vrouter-container-x86.26.2.2.tgz`。
5. 核對下載頁上的檔名與大小，確認與收到的檔案一致。
6. 若檔案詳細資料提供 checksum，使用相同演算法核對下載檔。

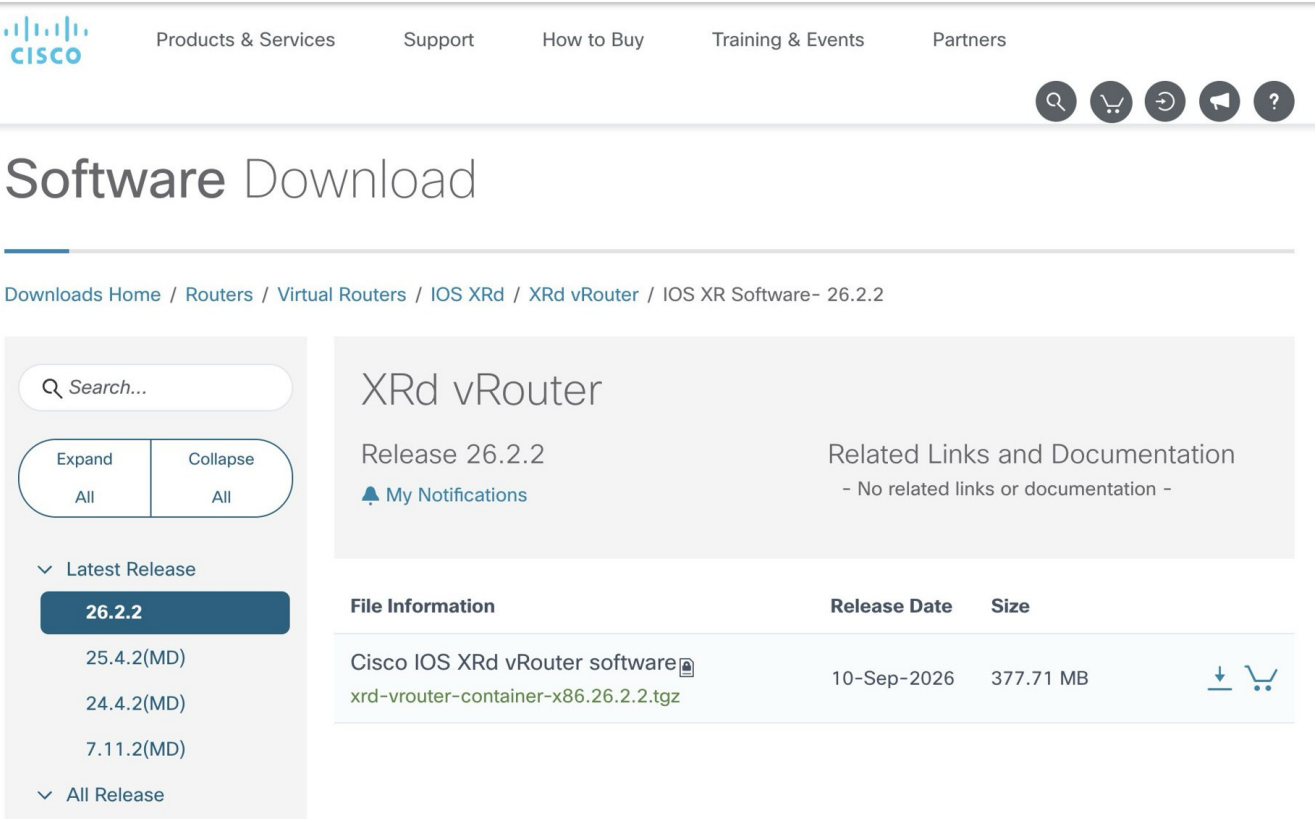


圖 1：這是公開的 Cisco 下載頁。畫面顯示 Release `26.2.2`、發行日期 `10-Sep-2026`、大小 `377.71 MB`。下載後核對這些資料。來源：software.cisco.com 的公開頁面。© Cisco 和 / 或其關係企業。

下載得到的是 Cisco container 軟體封存檔。後續步驟會取出其中的 Docker image archive，並將它載入自行建置的 Ubuntu 磁碟。

階段	檔案	下一步
Cisco 下載	`xrd-vrouter-container-x86.26.2.2.tgz`	列出內容並解壓
解壓後	`*.dockerv1.tgz`，實際名稱依封存檔內容	放入建置目錄的 `share/`
建置完成	`standalone.qcow2`	複製到 CML drop folder
CML 登錄	`xrd-vr-e1000-26-2-2.qcow2`	關聯到 Image Definition

3. 準備 Linux 建置主機

3.1 確認 KVM

執行位置：Linux 建置主機與 CML 主機，各執行一次。

```
uname -m
ls -l /dev/kvm
lscpu | grep -o -w -E 'ssse3|sse4_1|sse4_2|pdpe1gb|vmx|svm' | sort -u
```

預期看到 `x86_64`、`/dev/kvm`，以及 `ssse3`、`sse4_1`、`sse4_2` CPU 旗標。`vmx` 或 `svm` 表示 Intel 或 AMD 虛擬化能力。

本 repository 的腳本使用 1 GiB hugepages，CPU 需提供 `pdpe1gb`，且 hypervisor 必須將這項能力提供給 guest VM。缺少這項能力時，先檢查 host 與 VM 的 CPU 設定。節點啟動後，Console 中的 `Hugepagesize` 應為 `1048576 kB`；使用預設值時，`HugePages_Total` 應為 `1`。

CML 跑在 ESXi VM 上時，需將硬體虛擬化能力開放給 CML VM，以便執行 nested KVM。若沒有 `/dev/kvm`，先處理 hypervisor 設定，再繼續安裝。

3.2 準備建置工具

執行位置：專用的 Ubuntu Linux 建置主機。

```
command -v qemu-system-x86_64 qemu-img genisoimage isoinfo timeout
```

缺少工具時，在專用建置主機執行：

```
sudo apt-get update
sudo apt-get install -y qemu-system-x86 qemu-utils genisoimage
```

具備工具的 CML 主機也可用來建置。請在執行前確認資源足夠；CML 主機的套件變更需依其維護程序處理。

3.3 傳送來源檔案

將下列項目傳送到 Linux 建置主機的使用者家目錄：

- 本 repository 的 `build/`，放在 `~/xrd-vrouter-cml/build/`。
- Cisco 下載檔，放在 `~/xrd-vrouter-container-x86.26.2.2.tgz`。
- 從 Ubuntu 官方目錄 下載的 `noble-server-cloudimg-amd64.img`，放在家目錄。下載時依 Ubuntu 提供的 checksum 核對檔案。

可使用 SCP 或 SFTP 傳送。先在建置主機建立 `~/xrd-vrouter-cml/`。以下在操作電腦上、含有 `xrd-vrouter-cml/` 的目錄中執行。請先替換 `BUILD_USER` 與 `BUILD_HOST`：

```
scp -r xrd-vrouter-cml/build \
  BUILD_USER@BUILD_HOST:~/xrd-vrouter-cml/
```

其餘兩個下載檔也可使用 SCP 傳送到建置主機的家目錄。

3.4 建立建置目錄

執行位置：Linux 建置主機。

以下使用預設路徑 `/var/tmp/xrd-vrouter-lab`。若該路徑已有先前建置資料，請保留原資料，另選新的空目錄，並在後續指令及 `WORKDIR` 參數使用新路徑。

```
sudo install -d -o "$(id -un)" -g "$(id -gn)" \
  /var/tmp/xrd-vrouter-lab
mkdir -p /var/tmp/xrd-vrouter-lab/src \
  /var/tmp/xrd-vrouter-lab/share \
  /var/tmp/xrd-vrouter-lab/cisco
cp -R ~/xrd-vrouter-cml/build/. /var/tmp/xrd-vrouter-lab/src/
cp ~/noble-server-cloudimg-amd64.img /var/tmp/xrd-vrouter-lab/
```

3.5 解壓 Cisco 封存檔

先列出封存檔內容：

```
tar -tzf ~/xrd-vrouter-container-x86.26.2.2.tgz
```

將封存檔解壓到剛建立的 `cisco/` 目錄：

```
tar -xzf ~/xrd-vrouter-container-x86.26.2.2.tgz \
  -C /var/tmp/xrd-vrouter-lab/cisco
find /var/tmp/xrd-vrouter-lab/cisco -type f -name '*.dockerv1.tgz'
```

確認找到對應 XRd vRouter 的 Docker image archive。以下的 `ACTUAL_DOCKERV1_PATH` 必須換成上一個指令顯示的完整路徑：

```
cp ACTUAL_DOCKERV1_PATH \
  /var/tmp/xrd-vrouter-lab/share/ios-xrd-vrouter-26-2-2.tar.gz
gzip -t /var/tmp/xrd-vrouter-lab/share/ios-xrd-vrouter-26-2-2.tar.gz
```

此處只複製檔案並使用建置腳本的預設檔名，Docker image 的 tag 會在建置 VM 內處理。建置主機本身只需執行 QEMU。

若沒有找到 `*.dockerv1.tgz`，請先檢查封存檔內的 README 與檔案清單，確認 Docker image archive 的實際包裝格式，再繼續。請勿只靠更改副檔名判定檔案用途。

完成後，目錄應包含：

```
/var/tmp/xrd-vrouter-lab/
├── noble-server-cloudimg-amd64.img
├── cisco/                  # Extracted Cisco package
├── share/
│   └── ios-xrd-vrouter-26-2-2.tar.gz
└── src/
    ├── build-golden.sh
    ├── user-data
    ├── meta-data
    ├── setup.sh
    └── stub.sh
```

4. 建置並檢查 qcow2 磁碟

執行位置：Linux 建置主機。

4.1 執行建置

```
sudo bash /var/tmp/xrd-vrouter-lab/src/build-golden.sh
```

腳本會建立 32 GiB 的 qcow2 overlay，啟動建置 VM，安裝 Docker，載入 XRd image，設定啟動服務，並在完成後關機。VM 必須能透過 QEMU 的網路連線存取 Ubuntu 套件庫。

如需不同建置路徑或封存檔名稱，可明確指定參數。此版腳本會將 `XRD_ARCHIVE` 與 `XRD_TAG` 寫入 shell 設定檔；封存檔名請只用英文字母、數字、句點、底線與連字號，Docker tag 請使用有效格式，兩者均需避免空白及 shell 特殊字元。首次安裝可直接沿用下列預設值：

```
sudo WORKDIR=/var/tmp/xrd-vrouter-lab \
  XRD_ARCHIVE=ios-xrd-vrouter-26-2-2.tar.gz \
  XRD_TAG=ios-xrd-vrouter:26-2-2 \
  bash /var/tmp/xrd-vrouter-lab/src/build-golden.sh
```

參數	意義
`WORKDIR`	含有 cloud image、`share/` 與 `src/` 的建置目錄
`XRD_ARCHIVE`	`share/` 內 Docker image archive 的檔名
`XRD_TAG`	VM 內執行 XRd 時使用的 Docker tag

腳本最長等待建置 VM 執行 2700 秒。可在另一個終端機查看進度：

```
sudo tail -f /var/tmp/xrd-vrouter-lab/serial.log
```

安裝程序掛載共享目錄後，另有 `/var/tmp/xrd-vrouter-lab/share/setup.log` 可供檢查。

4.2 確認建置結果

成功的建置應符合以下條件：

- `serial.log` 中有 `XRD_GOLDEN_READY`。
- 建置指令輸出 `QEMU_RC:0` 與 `GOLDEN_OK`。
- 指令正常結束，並產生 `standalone.qcow2` 及其 SHA-256。

```
sudo grep XRD_GOLDEN_READY /var/tmp/xrd-vrouter-lab/serial.log
sudo qemu-img info /var/tmp/xrd-vrouter-lab/standalone.qcow2
sudo qemu-img check /var/tmp/xrd-vrouter-lab/standalone.qcow2
sudo sha256sum /var/tmp/xrd-vrouter-lab/standalone.qcow2
```

`qemu-img info` 應顯示 qcow2，且沒有 `backing file`；`qemu-img check` 應無錯誤。記下 SHA-256，稍後用來核對傳到 CML 的檔案。

上傳使用的檔案是 `standalone.qcow2`。建置目錄中的 `golden.qcow2` 仍依賴 Ubuntu cloud image；`qemu-img convert` 產生的 `standalone.qcow2` 已包含完整磁碟內容。

若出現 `GOLDEN_FAIL` 或既有檔案提示，先保留 `serial.log` 與 `share/setup.log`。修正原因後使用新的建置目錄。腳本會拒絕覆寫已有的 `seed.iso`、`golden.qcow2`、`standalone.qcow2` 或 `serial.log`。

5. 在 CML 登錄自訂節點

先匯入 Node Definition，再建立 Image Definition。兩者的 ID 必須與第 1 節一致。

5.1 準備 Node Definition

執行位置：操作電腦的文字編輯器。

1. 開啟 `definitions/xrd-vr-e1000-node.yaml` 的工作副本。
2. 找到 `first-boot.cfg` 內的 `username lab`。
3. 將 `secret 0 CHANGE_ME_BEFORE_IMPORT` 中的占位符換成自行設定的密碼。
4. 儲存工作副本，保留 YAML 縮排與完整檔案結構。

`secret 0` 後方是明文輸入值，因此修改後的 YAML 會含有密碼。請妥善保存，分享範本前移除實際密碼。

也可以輸入雜湊後的 secret，避免 YAML 內出現明文。在 Linux 主機用 `openssl passwd -6` 產生雜湊，再以 `secret 10 <hash>` 輸入。第一次啟動後，用 `show running-config` 檢查該項目，並測試登入。

以下欄位已由定義檔設定，首次安裝保留預設值：

欄位	值或用途
`id`	`xrd-vr-e1000`
`sim.linux_native.libvirt_domain_driver`	`kvm`
`sim.linux_native.driver`	`iosxrv9000`，用於 CML 主控台與設定整合
`sim.linux_native.disk_driver`	`virtio`
`sim.linux_native.nic_driver`	`e1000`
`sim.linux_native.cpus`	`2`
`sim.linux_native.ram`	`7168` MiB
`boot.timeout`	`1800` 秒
`device.interfaces.default_count`	`4`，包含管理介面
`device.interfaces.min_count`	`2`，管理介面加至少一個 data plane 介面
`inherited`	完整保留 CPU、RAM 等繼承設定

Node Definition 內還有三個 provisioning 檔案：

檔案	在 CML 的可編輯狀態	用途
`first-boot.cfg`	可編輯	第一次初始化 IOS XR 的設定與帳號
`xrd.env`	可編輯	XRd 環境變數
`guest-init.sh`	不可編輯	VM 開機時辨識 PCI 網卡並啟動 XRd

保留 `xrd.env` 的預設內容：

```
XR_VROUTER_DP_HUGEPAGE_MB=1024
XR_VROUTER_DP_MAIN_TUNE=100,250,500,80,100
```

5.2 匯入 Node Definition

執行位置：CML Web UI，使用具管理權限的帳號。

1. 開啟 **Tools → Node and Image Definitions**。
2. 切換至 **Node Definitions**。
3. 使用匯入功能，上傳剛才設定好密碼的完整 YAML。
4. 確認清單中出現 `IOS XRd vRouter (e1000)`，ID 為 `xrd-vr-e1000`。

若同名定義已存在，先確認它是否已由其他 lab 使用。既有安裝可直接檢查定義及映像關聯；更新共用定義前，需確認受影響的節點。

5.3 傳送 qcow2 到 CML

先將 `standalone.qcow2` 傳到 CML 管理帳號的家目錄。以下範例在建置主機、一般建置帳號的 shell 中執行，先替換 `CML_USER` 與 `CML_HOST`。

建置由 root 執行，先將這個新產生的輸出檔交給建置帳號，再使用該帳號的 SSH 設定傳送：

```
sudo chown "$(id -un):$(id -gn)" \
  /var/tmp/xrd-vrouter-lab/standalone.qcow2
scp /var/tmp/xrd-vrouter-lab/standalone.qcow2 \
  CML_USER@CML_HOST:~/standalone.qcow2
```

接著 SSH 登入 CML 主機，確認目的檔案與映像目錄尚未存在：

```
sudo ls -ld /var/local/virl2/dropfolder
sudo test ! -e \
  /var/local/virl2/dropfolder/xrd-vr-e1000-26-2-2.qcow2 \
  && echo DROPFILE_FREE
sudo test ! -e \
  /var/lib/libvirt/images/virl-base-images/xrd-vr-e1000-26-2-2 \
  && echo IMAGE_DIRECTORY_FREE
```

看到 `DROPFILE_FREE` 與 `IMAGE_DIRECTORY_FREE` 後，再複製磁碟。任一標記沒有出現時，先查明既有檔案的用途。

```
sudo install -o virl2 -g virl2 -m 664 ~/standalone.qcow2 \
  /var/local/virl2/dropfolder/xrd-vr-e1000-26-2-2.qcow2
sudo sha256sum \
  /var/local/virl2/dropfolder/xrd-vr-e1000-26-2-2.qcow2
```

SHA-256 必須與建置主機的 `standalone.qcow2` 相同。這裡只把磁碟放入 drop folder；正式的映像目錄由 CML 建立。

5.4 建立 Image Definition

執行位置：CML Web UI。

1. 回到 **Tools → Node and Image Definitions**。
2. 在 drop folder 的可用映像清單找到 `xrd-vr-e1000-26-2-2.qcow2`。若清單尚未更新，重新整理頁面。
3. 使用該磁碟建立 Image Definition，依下表填入欄位。各 CML 版本的按鈕名稱可能不同，請以欄位與關聯值核對。
4. 儲存後，確認 Image Definition 已關聯到 `xrd-vr-e1000`。

欄位	值
ID	`xrd-vr-e1000-26-2-2`
Label	`XRd vRouter 26.2.2 (e1000)`
Node Definition	`xrd-vr-e1000`
Disk Image	`xrd-vr-e1000-26-2-2.qcow2`

`definitions/xrd-vr-e1000-26-2-2-image.yaml` 提供相同欄位值。映像檔本身仍需依前一步傳送到 CML。

儲存成功後，CML 會將磁碟從 drop folder 移到正式目錄。在 CML 主機檢查：

```
sudo ls -lh \
  /var/lib/libvirt/images/virl-base-images/xrd-vr-e1000-26-2-2/
sudo sha256sum \
  /var/lib/libvirt/images/virl-base-images/xrd-vr-e1000-26-2-2/xrd-vr-e1000-26-2-2.qcow2
```

確認磁碟存在、SHA-256 相同，即完成映像登錄。

5.5 選用：vmxnet3 版本

本指南預設使用 e1000 版本。本 repository 另有使用 vmxnet3 網卡的版本，需要時才使用。

1. 替換密碼占位符後，匯入 `definitions/xrd-vr-vmxnet3-node.yaml`。
2. 以 `xrd-vr-vmxnet3-26-2-2.qcow2` 為檔名，再複製一次 `standalone.qcow2`。步驟與 5.3 節相同。
3. 為 Node Definition `xrd-vr-vmxnet3` 建立 Image Definition `xrd-vr-vmxnet3-26-2-2`。

這個版本的 data plane 介面名稱是 `TenGigE0/0/0/N`，不是 `GigabitEthernet0/0/0/N`。所有設定與連線描述中的介面名稱都要更換，包括 `examples/` 的檔案。10G 介面的預設 OSPF cost 比 1G 介面低。需要與 1G 介面相同的 cost 時，在介面下加入 `bandwidth 1000000`。

6. 建立節點並確認啟動

執行位置：CML Web UI 與節點 Console。

1. 建立一個專用的安裝確認 lab。
2. 加入 `IOS XRd vRouter (e1000)`。
3. 確認節點使用 Image Definition `xrd-vr-e1000-26-2-2`。
4. 保留 2 vCPU、7168 MiB RAM 與預設四個介面。
5. 在節點設定中確認 `first-boot.cfg` 的帳號與 `xrd.env` 預設值。
6. 啟動節點，開啟第一個序列 Console。
7. 等待 XR CLI 可用，再使用先前設定的 `lab` 帳號登入。

單一節點開機確認可在沒有鏈路的情況下進行。節點仍需保留管理介面及至少一個 data plane 介面，供啟動腳本辨識。

Console 預期依序出現下列訊息，其中 `<netdev>`、`<PCI address>` 為實際辨識結果：

```
XRD_GUEST_INIT_START kvm
XRD_GUEST_NOIOMMU Y
```

```
XRD_GUEST_ENV XR_VROUTER_DP_HUGEPAGE_MB=1024 ...
XRD_GUEST_INIT_RUN mgmt=<netdev>(<PCI address>) data=pci:<PCI address>
ios con0/RP0/CPU0 is now available
```

以 `is now available` 及可登入的 XR CLI 作為就緒判斷。CML 的 `BOOTED` 狀態可能較早出現；請繼續觀察 Console，開機時間會受主機負載影響。

在 **XR CLI** 執行：

```
show version
show ipv4 interface brief
```

預期結果：

- `show version` 顯示 `Cisco IOS XR Software, Version 26.2.2 LNT`。
- 介面清單包含 `MgmtEth0/RP0/CPU0/0`。
- 預設四個介面時，清單包含 `GigabitEthernet0/0/0/0` 至 `GigabitEthernet0/0/0/2`。

此時介面尚未設定 IP，或因預設 shutdown 而顯示 down，均可由後續 lab 設定處理。完成上述檢查，即可開始在 CML 使用這個自訂節點。

6.1 選用：用範例檢查轉送

執行位置：CML 網頁介面與各節點的 console。

這項測試使用兩個 XRd 節點與一個 IOS 節點，檢查 data plane、LLDP 與 CDP。

1. 建立 lab，加入節點 `VR1`、`VR2`（`IOS XRd vRouter (e1000)`）與節點 `IOS`。
2. 連接 `VR1` GigabitEthernet0/0/0/0 與 `VR2` GigabitEthernet0/0/0/0。
3. 連接 `VR1` GigabitEthernet0/0/0/1 與 `IOS` GigabitEthernet0/0。
4. 在三個節點分別載入 `examples/VR1.cfg`、`examples/VR2.cfg`、`examples/IOS.cfg`。先替換 `CHANGE_ME_BEFORE_IMPORT`。
5. 啟動節點，等待 XR CLI 可用。

在 `VR1` 執行：

```
ping 10.10.10.2
ping 10.10.20.2
show lldp neighbors
show cdp neighbors
```

兩個 ping 都必須成功。`VR1` 的 LLDP 與 CDP 必須列出 `VR2`，CDP 另須列出 `IOS` 節點。在 `VR2` 執行 `ping 10.10.10.1` 也必須成功。

7. 開機設定與日常使用

7.1 設定保存

XR 設定保存在節點磁碟內的 Docker volume `xr-storage`。在 XR CLI 提交的設定會在 CML stop/start 後保留。

`first-boot.cfg` 用於第一次初始化。已有持久設定的節點，日常修改應在 XR CLI 執行並 `commit`。若要驗證新的 Day-0 範本，可建立新節點。CML 的 wipe 會清除節點 overlay 與其中的 XR 設定，再從基底磁碟重新初始化；執行前需另存所需設定。

7.2 Hugepages 與環境變數

建置腳本在 **guest VM** 的 GRUB 預留三個 1 GiB hugepages。節點啟動時，`guest-init.sh` 根據 `XR_VROUTER_DP_HUGEPAGE_MB=1024` 釋放多餘頁面，預設保留 1 GiB。7168 MiB RAM 已包含這些 hugepages。

Hugepage 與 socket sysctl 調整均由 guest 內的腳本處理。安裝者可保留預設值，CML 主機沿用原有設定。

`xrd.env` 內另有兩行選用設定，它們控制 guest，不是 XRd。`GUEST_SOCKET_SYSCTL=0` 會略過 socket sysctl 參數。`GUEST_THP=never`、`madvise` 或 `always` 會設定 guest 內的 transparent hugepage。啟動腳本也會把這兩行傳給容器，我們沒有檢查 XRd 如何處理它們。

XRd 26.2.2 的 `XR_VROUTER_DP_MAIN_TUNE` 需使用五個值。若自行增加環境變數，請依對應 XRd 版本核對名稱，並從 Console 檢查是否有 `Invalid environment variable(s)`。

7.3 搬到另一台 CML

另一台 CML 需要同一套 Node Definition、Image Definition 與 qcow2。匯出 lab YAML 只包含 lab 的節點及設定，安裝自訂節點時仍需準備這三項。

複製已建好的基底 qcow2 並核對 SHA-256 後，可在目標 CML 重做第 5 節與第 6 節。每台主機都需具備可用的 KVM，並遵守軟體授權條款。

更新 XRd 版本時，使用新的建置目錄、磁碟檔名與 Image Definition ID。既有節點依賴原本的基底磁碟，請保留該磁碟直到所有相依節點完成移轉。

8. 安裝故障排查

現象	檢查位置與原因線索	處理方式
`/dev/kvm` 不存在	Linux 建置主機或 CML 主機；虛擬化能力未提供給 VM	先確認 CPU 虛擬化與 hypervisor 的 nested virtualization 設定
1 GiB hugepage 路徑不存在或配置失敗	Console；CPU 的 `pdpe1gb` 能力與 guest 記憶體	確認 hypervisor 將能力提供給 guest，並核對 7168 MiB RAM 與預設環境變數
找不到 `*.dockerv1.tgz`	Cisco 下載封存檔的內容清單	確認下載的產品與版本，閱讀包內 README，找出真正的 Docker image archive
`STOP: existing build artifact`	建置目錄已有磁碟、ISO 或 log	保留資料，改用新的建置目錄
`GOLDEN_FAIL`、建置逾時	`serial.log`、`share/setup.log`	檢查 Ubuntu 套件庫連線、磁碟空間及 `docker load` 錯誤；修正後重新建置
載入 image 或設定 tag 失敗	`setup.log` 中的 `Loaded image:` 與 Docker 錯誤	核對輸入檔是 Docker image archive，並確認 image 可對應 `XRD_TAG`
Image Definition 建立時出現 `Failed to create folder`	正式映像目錄已存在	先查明目錄與既有 Image Definition 的關聯；僅對確認未使用的安裝殘留資料另行處理
CML 清單找不到 qcow2	drop folder 的檔名、擁有者與權限	核對第 5.3 節，確認 `virl2:virl2` 與模式 `664`，再重新整理頁面
無法建立節點	Node Definition 的完整性與 `inherited` 區段	重新核對本 repository 的 YAML，保留全部欄位與正確縮排
`XRD_GUEST_INIT_FAIL`	Console 中該標記後的詳細訊息	確認有管理與 data plane 網卡，以及設定光碟中的三個 provisioning 檔案
`Invalid environment variable(s)`	節點的 `xrd.env`	回復本說明書的兩個預設變數，再逐一檢查新增變數
`unsupported PCI IDs`	Node Definition 的 `nic_driver`	使用本 repository 的 `e1000` 定義
`XRd is expected to require 5Gi of RAM`	節點 RAM 與 guest 的 hugepage 設定	核對 7168 MiB RAM，並保留預設的 `XR_VROUTER_DP_HUGEPAGE_MB=1024`

現象	檢查位置與原因線索	處理方式
`BOOTED` 後仍無 XR CLI	Console 的 guest 與 XRd 啟動訊息	等待 `is now available`；若訊息停止或反覆重啟，記錄最早的錯誤再排查
修改 `first-boot.cfg` 後登入資料仍是舊值	節點已有 `xr-storage` 持久設定	使用原帳號登入修改，或建立新節點驗證新的初始化設定

9. 安裝完成條件與資料來源

完成以下項目即可交付自訂節點：

- CML 有 `xrd-vr-e1000` Node Definition。
- CML 有 `xrd-vr-e1000-26-2-2` Image Definition，並關聯正確磁碟。
- 建置主機與 CML 正式目錄中的 qcow2 SHA-256 一致。
- 新節點可啟動，並可用自行設定的帳號登入 XR CLI。
- XR CLI 顯示版本 26.2.2，以及管理介面與預期的 data plane 介面名稱。

本說明書的安裝參數來自本 repository 的 `build/` 與 `definitions/`。

參考資料：

- Cisco Software Download：XRd vRouter 26.2.2
- Cisco CML：Preparing a Custom Image
- Ubuntu 24.04 cloud image
- Cisco IOS XRd Data Sheet
- ios-xr/xrd-tools

10. 使用範圍、授權與商標

- **僅限實驗環境。** Guest 以 `--privileged` 且不使用 AppArmor 限制的方式啟動 XRd 容器。請只在實驗環境使用，不要用於正式環境。
- **不修改 XRd。** 腳本原樣載入 Cisco XRd 映像，只在外面加上 Linux guest、啟動腳本與 CML 定義檔。本 repository 不含 Cisco 軟體，也不含磁碟映像。
- **支援。** 這是獨立的作品，Cisco 不提供支援，我們也不提供個別支援。
- **授權。** 腳本、定義檔與範例使用 MIT License（`LICENSE`）。MIT License 不涵蓋 Cisco 軟體、Cisco 商標與 Cisco 文件。文件（README 與 PDF）使用 CC BY-NC-ND 4.0（`LICENSE-docs`）。Copyright 2026 quintus-lab。
- **商標。** Cisco、Cisco Modeling Labs、CML、IOS XR 與 XRd 是 Cisco 和 / 或其關係企業在美國及其他國家的商標或註冊商標。其他名稱屬於各自的所有人。本作品與 Cisco 無關聯。